

# PIC

## Verification Specification

Reproducible stimuli, checks and waveform evidence

---

|               |                                                    |
|---------------|----------------------------------------------------|
| Document      | PIC — Verification Specification                   |
| Document type | Verification specification                         |
| Subject       | Reproducible stimuli, checks and waveform evidence |
| Revision      | 2.1                                                |
| Date          | 21 September 2026                                  |
| Author        | Bunea Cosmin-Andrei                                |
| Status        | Released for review                                |

---

## Contents

|          |                                                       |           |
|----------|-------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                   | <b>2</b>  |
| 1.1      | Purpose of the document . . . . .                     | 2         |
| 1.2      | Validity of the document . . . . .                    | 2         |
| 1.3      | Terms and abbreviations . . . . .                     | 2         |
| 1.4      | Relationship with other documents . . . . .           | 3         |
| 1.5      | Notes on the notation used . . . . .                  | 3         |
| <b>2</b> | <b>Verification methodology</b>                       | <b>4</b>  |
| 2.1      | Overview and acceptance . . . . .                     | 4         |
| 2.2      | Implementation strategy and reference model . . . . . | 5         |
| <b>3</b> | <b>Verification environment</b>                       | <b>6</b>  |
| 3.1      | Overview . . . . .                                    | 6         |
| 3.2      | Control and drivers . . . . .                         | 7         |
| 3.3      | Monitors and checkers . . . . .                       | 8         |
| 3.4      | Simulation environment setup . . . . .                | 9         |
| 3.4.1    | Structure . . . . .                                   | 9         |
| 3.4.2    | Scripts . . . . .                                     | 9         |
| 3.4.3    | Setup and configuration . . . . .                     | 10        |
| <b>4</b> | <b>Annex</b>                                          | <b>11</b> |
| 4.1      | Verification plan . . . . .                           | 11        |
| 4.2      | Critical sequences . . . . .                          | 12        |
| 4.3      | External AXI reference . . . . .                      | 13        |

# 1 Introduction

## 1.1 Purpose of the document

This specification describes the simulation strategy, stimulus, reference models and acceptance checks of the programmable interrupt controller. It is intended for hardware designers, verification engineers and software developers integrating the block.

It defines the behaviour that the RTL must show at its interfaces.

## 1.2 Validity of the document

The documented configuration is the RTL in `cpu/hdl` and the associated CPU/PIC and SoC simulation environments. Interface adaptations from the supplied briefs are identified in the integration section.

Revision 2.1 of 21 September 2026 describes the delivered state of that RTL.

## 1.3 Terms and abbreviations

| Term      | Meaning                                             |
|-----------|-----------------------------------------------------|
| AXI4-Lite | Single-beat AXI read/write channels.                |
| CSR       | Control and status register.                        |
| DUT       | Device under test.                                  |
| EOI       | End of interrupt; completes one service level.      |
| IRQ       | Interrupt request.                                  |
| NBA       | Nonblocking-assignment update region in simulation. |
| PIC       | Programmable interrupt controller.                  |
| SVA       | SystemVerilog Assertions.                           |

## 1.4 Relationship with other documents

The documents below are the ones this specification is derived from or read with.

| Reference                                                  | Version / role                                                                |
|------------------------------------------------------------|-------------------------------------------------------------------------------|
| Programmable Interrupt Controller with Advanced Scheduling | Supplied project draft; functional requirements and implementation decisions. |
| Spec table of contents and templates.pdf                   | Supplied document structure and verification-plan template.                   |
| Design Specification — PIC                                 | Companion specification, revision 2.1, 21 September 2026.                     |
| ARM AMBA AXI protocol                                      | IHI0022E, AXI4-Lite.                                                          |
| IEEE 1364-2005                                             | Verilog RTL.                                                                  |

## 1.5 Notes on the notation used

Signal and register names follow the RTL. Addresses and waveform bus values are hexadecimal; time is in ns and widths are in bits. Register offsets are byte offsets; CSR addresses are instruction-encoded indices.

| Notation                                        | Meaning                                                                |
|-------------------------------------------------|------------------------------------------------------------------------|
| Fixed-width name                                | A Verilog module, port, file or script name, spelled as in the source. |
| Signal suffix <code>_i</code> / <code>_o</code> | Module input / output, seen from inside that module.                   |
| <code>[a:b]</code>                              | Bit range, most significant bit first.                                 |
| Source slot                                     | One of the 16 arbitration identities; x or n denotes its index 0–15.   |
| posedge +1 ns                                   | A testbench stimulus time, one nanosecond after the sampling edge.     |

## 2 Verification methodology

### 2.1 Overview and acceptance

A run either passes or fails; this section states what that verdict covers. The first table gives the acceptance conditions applied to every test and the limits of what simulation shows; the second gives the runs behind the results quoted in this document.

| Item               | Contract                                                                                              |
|--------------------|-------------------------------------------------------------------------------------------------------|
| Requirement source | Programmable Interrupt Controller with Advanced Scheduling (original PIC brief).                      |
| Device under test  | pic and axi_lite_slave.                                                                               |
| Functional verdict | Every expected value and response must match; errors=0 and test_done=1 are required.                  |
| Protocol verdict   | VALID/payload stability, response ordering and outstanding transaction limits; clocked at posedge.    |
| Failure handling   | Mismatch, timeout, incomplete test, assertion failure or coverage-gate failure returns a failing run. |
| Limits of evidence | Directed simulation and reference-model checks, not a formal proof or a synthesis/timing result.      |
| Simulator / flow   | Recorded evidence (18–21 September 2026)                                                              |
| ModelSim           | 22 CPU/PIC configurations; zero mismatches.                                                           |
| Verilator –assert  | 16 CPU/PIC benches and the CPU functional-coverage gate.                                              |
| Integrated SoC     | 25 ModelSim configurations and 16 assertion-enabled benches.                                          |
| PULP comparison    | Decoder sequence plus 96 register-slave strobe/order/stall cases and RO/unmapped/reset cases.         |

## 2.2 Implementation strategy and reference model

Directed tests exercise individual features and simultaneous events. Program/reference tests check end-to-end results. Protocol checks observe accepted transfers; state checks sample after nonblocking updates.

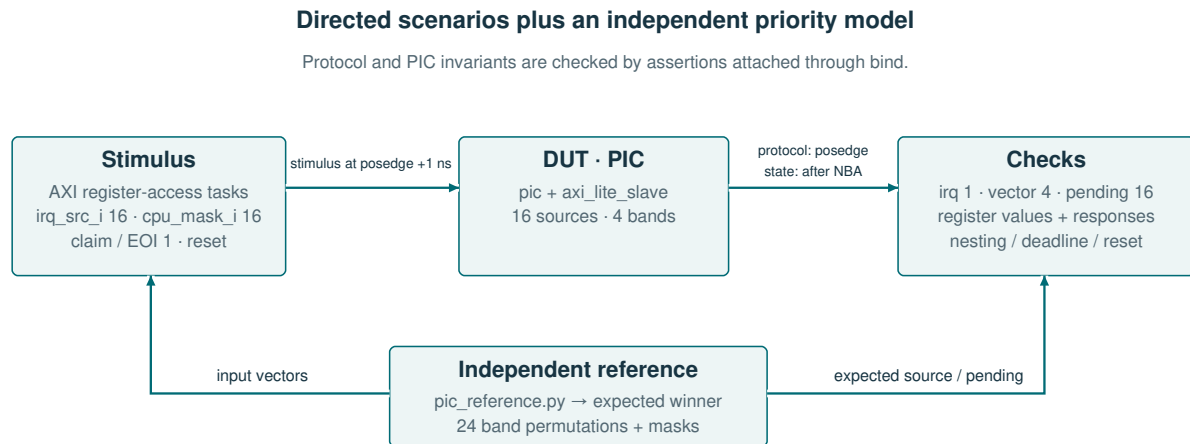
The independent priority model enumerates all 24 band-urgency permutations with source, enable and CPU masks. Expected winner and pending set are compared with the RTL; directed tests add temporal behaviors such as nesting and deadline escalation.

The verification plan records an ID, scenario, expected result, status and test/checker. Its detailed cases appear in the annex; the environment below identifies the components that execute and check them.

## 3 Verification environment

### 3.1 Overview

Register-access tasks configure the PIC and drive source, mask and claim/EOI events. The independent model supplies expected winners and pending masks; the checkers also inspect register responses and service-state transitions.



**Figure 1:** Test stimuli, DUT and independent checking paths.

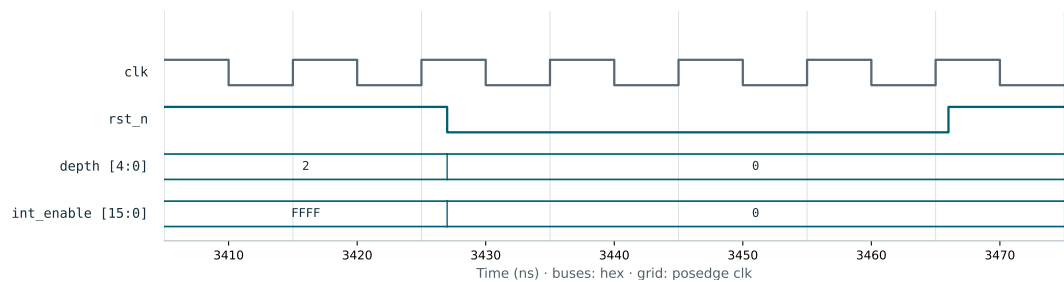
| Component     | File / role                                                                                             |
|---------------|---------------------------------------------------------------------------------------------------------|
| Clock / reset | ck_rst_tb.v: shared generator; dedicated reset benches can drive a reset pulse between edges.           |
| Bus stimulus  | tb_axil_master.vh: register-access tasks; CPU programs supply real load/store requests at system level. |
| Memory model  | axi_lite_mem_model.v: configurable response delay and repeatable backpressure.                          |
| Reference     | pic_reference.py: expected priority winner and pending mask.                                            |
| Verdict       | tb_check.vh: explicit mismatch count, completion and fatal failure.                                     |
| Protocol      | axi_lite_monitor.v for ModelSim; axi_lite_sva.sv under Verilator.                                       |

### 3.2 Control and drivers

Stimulus is driven one nanosecond after the sampling edge, so that a test never races the design it is checking. The table gives the timing rule for each driven event and the reason for it; the waveform shows the reset case, the one event placed off the clock grid.

| Signal / event           | Timing rule                                                                          | Reason                                                            |
|--------------------------|--------------------------------------------------------------------------------------|-------------------------------------------------------------------|
| clk                      | 10 ns period in these traces; rising edges are 5, 15, 25... ns.                      | Defines the handshake observation points.                         |
| Shared bus / claim tasks | Drive 1 ns after posedge, in a separate statement. Hold until the sampling edge.     | Avoid blocking-assignment races with DUT active/NBA evaluation.   |
| Handshake observation    | Sample VALID && READY at posedge before changing stimulus.                           | Records the transfer that the DUT accepted.                       |
| Reset assertion          | Dedicated reset tests assert at posedge +2 ns, check at +3 ns.                       | The check occurs before either the next falling or rising edge.   |
| Reset release            | Explicit nanosecond delays, off the clock grid; shared generator releases at 123 ns. | Exercises asynchronous stimulus without a coincident clock event. |

The controller starts with active nested service and enabled sources. Reset clears both depth and INT\_ENABLE between clock edges; the test checks the state one nanosecond after assertion.



**Figure 2:** Reset from active state at posedge +2 ns; cleared state is checked at +3 ns.

| Formatting convention | Code                                                                              |
|-----------------------|-----------------------------------------------------------------------------------|
| Separate statements   | @(posedge clk); followed on the next line by #1; and then the signal assignments. |
| Scope of #1           | Applies to testbench scheduling only.                                             |

### 3.3 Monitors and checkers

Checking is split between bus protocol and functional result, so that a failure points at one of the two. The first table lists each checker and what it observes; the second shows how each one is kept from passing without having seen traffic.

| File / mechanism       | What it checks                                                                                                                                                               |
|------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| axi_lite_monitor.v     | Passive ModelSim protocol checks and transaction counts; valid traffic must occur.                                                                                           |
| axi_lite_sva.sv        | Channel stability, legal response codes, request/response ordering, one outstanding transaction per direction.                                                               |
| pic_sva.sv             | Offer eligibility, CPU mask, priority threshold, depth transitions, mutual exclusion and empty EOI.                                                                          |
| rv32i_bind_core_sva.sv | bind attaches checker instances to RTL module types. The checkers read DUT signals; they drive nothing and add no port.                                                      |
| rv32i_bind_sva.sv      | Attaches the nominal CPU functional-coverage model; coverage is separate from the shared assertion bind.                                                                     |
| run_verilator.sh       | Compiles assertions with <code>-assert</code> and fails the run on an assertion failure. ModelSim ASE runs the procedural monitors instead; the SVA execute under Verilator. |

| Non-vacuous check  | Evidence                                                                                             |
|--------------------|------------------------------------------------------------------------------------------------------|
| Valid traffic      | Read/write counts and exercised tests prevent a disconnected monitor from being counted as coverage. |
| Preconditions      | RO-write test first creates nonzero state; async-reset test first creates pending/nonzero state.     |
| Negative scenarios | Watch an interval for forbidden offers/claims; do not infer absence from one sample.                 |

### 3.4 Simulation environment setup

The environment is a fixed folder layout plus a small number of named scripts. The tables give that layout, the exact command for each step in the order it is run, and the tools each step needs.

#### 3.4.1 Structure

| Folder                  | Contents                                                                                     |
|-------------------------|----------------------------------------------------------------------------------------------|
| cpu/hdl                 | Synthesizable RTL: the CPU modules, the PIC, the timer and the AXI4-Lite register front-end. |
| cpu/debug/hdl           | Testbenches, bus masters and the configurable memory and responder models.                   |
| cpu/debug/sva           | Assertion modules and the bind files that attach them to RTL module types.                   |
| cpu/debug/sim           | Run scripts, file lists, reference models, generated programs and run logs.                  |
| soc/hdl                 | Integration fabric: address decode, arbitration, AXI4-Full to Lite conversion and RAM.       |
| soc/debug/hdl, sva, sim | The same split as above, applied to the integrated SoC.                                      |
| cpu/docs                | Specification sources, generated figures and the waveform windows.                           |

#### 3.4.2 Scripts

| Step | Working directory            | Command / script                     | Result                                                 |
|------|------------------------------|--------------------------------------|--------------------------------------------------------|
| 1    | Repository root              | make asm                             | Regenerate programs and independent reference vectors. |
| 2    | cpu/debug/sim                | vsim -c -do "do regress.do; quit -f" | Compile and run 22 CPU/PIC configurations.             |
| 3    | cpu/debug/sim (Bash / WSL)   | bash run_verilator.sh                | Lint, functional coverage and 16 benches with SVA.     |
| 4    | soc/debug/sim                | vsim -c -do "do regress.do; quit -f" | 25 integration configurations.                         |
| 5    | Repository root (Bash / WSL) | bash soc/debug/sim/run_verilator.sh  | SoC lint and 16 assertion-enabled benches.             |
| 6    | soc/debug/sim (Bash / WSL)   | bash run_pulp_compare.sh             | Pinned external decoder/register comparison.           |
| 7    | cpu/debug/sim                | vsim -c -do "do waves.do"            | Run 8 checked benches and export VCD traces.           |
| 8    | Repository root              | python cpu/docs/tools/waveforms.py   | SVG/PDF/PNG figures and source-window manifest.        |

### 3.4.3 Setup and configuration

| Tool / setup   | Requirement                                                                                   |
|----------------|-----------------------------------------------------------------------------------------------|
| ModelSim       | vsim/vlog on PATH; run from the named simulation directory so file lists resolve.             |
| Verilator      | Verilator 5.050 used for the recorded run; C++ build tools, Bash and Python 3 are required.   |
| PULP           | Git and network only for the initial checkout; explicit revision pins recorded in the runner. |
| Wave rendering | Python with matplotlib; raw VCD files produced by waves.do.                                   |
| Configuration  | Latency/stall overrides are read back after elaboration; seeds are fixed for repeatability.   |

## 4 Annex

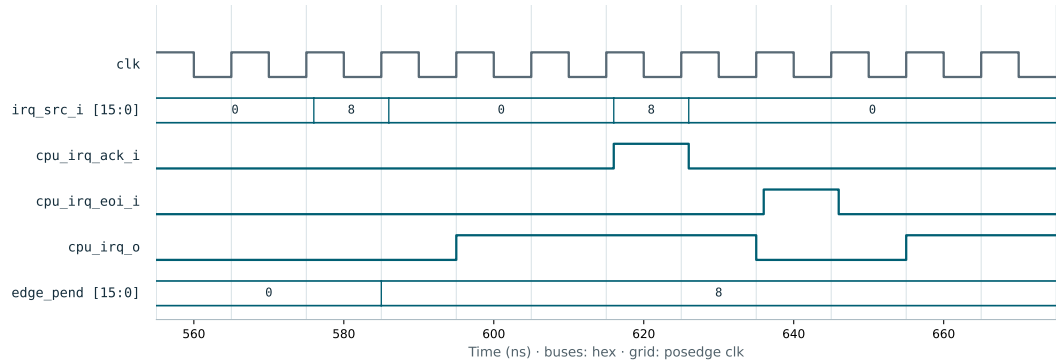
### 4.1 Verification plan

PASS indicates the recorded regression completed with its expected results and zero checker errors. Each row identifies the test that exercises the listed behavior.

| ID     | Title               | Expected result / scope                                                                                                         | Status | Test / checker                       |
|--------|---------------------|---------------------------------------------------------------------------------------------------------------------------------|--------|--------------------------------------|
| PIC-01 | Feature sequences   | Level/edge sources, priority bands/intra-band/ties, nesting, limit, spurious, escalation, SW trigger and access response codes. | PASS   | pic_tb_feature                       |
| PIC-02 | Priority reference  | Independent priority model: all 24 urgency permutations, request/enable/CPU masks and ties.                                     | PASS   | pic_tb_reference                     |
| PIC-03 | Simultaneous events | Masked source cannot starve another; fresh edge coincident with claim; reordered-band escalation; trigger-key strobes.          | PASS   | pic_tb_sched                         |
| PIC-04 | Asynchronous reset  | All per-source/global defaults, loaded-state reset before next clock edge, reset with AW-only/W-only/B-held/R-held, recovery.   | PASS   | pic_tb_reset                         |
| PIC-05 | Read-only registers | Write every RO register from a nonzero hardware state; expect SLVERR and unchanged value.                                       | PASS   | pic_tb_ro                            |
| PIC-06 | Status transitions  | PEND, ACTIVE, ESC, SPUR, EFF_BAND and timer fields; exact escalation transitions and complete source life cycle.                | PASS   | pic_tb_status                        |
| PIC-07 | CPU integration     | CPU programs the real PIC; claim/EOI order, masks, pending and interrupt handling across LSU stalls.                            | PASS   | rv32i_tb_cpu_axi                     |
| PIC-08 | SoC integration     | Interrupt source routing, nesting and escalation in the integrated SoC.                                                         | PASS   | soc_tb_pic_sources / nest / escalate |

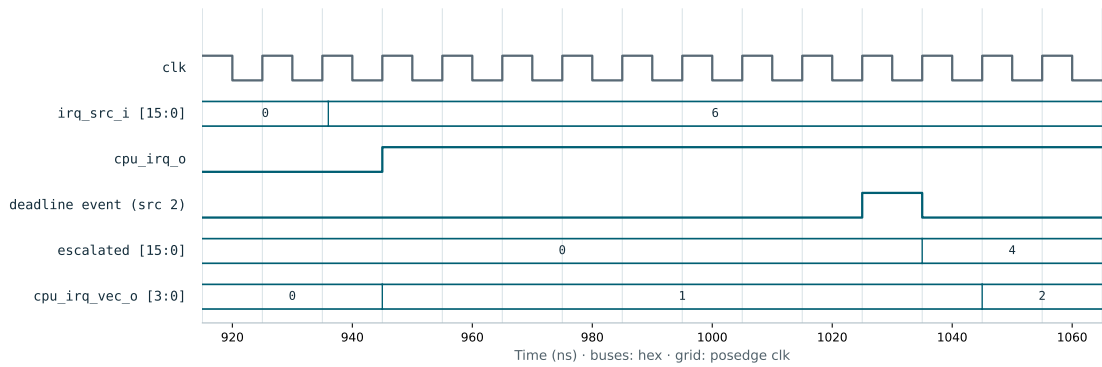
## 4.2 Critical sequences

A second source edge is sampled with the claim of the first event. The edge latch remains set, and the source is offered again after EOI.



**Figure 3:** New edge coincident with claim: the edge latch remains set.

Source 1 initially wins the tie. Source 2 reaches its ten-cycle deadline; the escalation event changes its effective priority, and the registered offer changes to source 2 on the following edge.



**Figure 4:** Ten-cycle deadline: escalation event, then registered source 2 offer under reordered band urgency.

| Scenario                     | Check                                                              |
|------------------------------|--------------------------------------------------------------------|
| New edge + claim             | Old event is consumed, new event is retained.                      |
| Band reordering              | Escalation follows programmed urgency, not numeric band order.     |
| Nest limit + eligible source | Offer suppressed and overflow logged; existing stack is preserved. |

### 4.3 External AXI reference

The AXI4-Lite front-end is compared against a published implementation to check the interpretation of the protocol. The first table states what was compared and on what evidence; the second names every difference found and what it means.

| Compared block                     | Shared contract                                                           | Evidence                                                                                           |
|------------------------------------|---------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------|
| soc_axi_lite_dec vs axi_lite_demux | Two mapped targets, MaxTrans=1; independent progress and response timing. | Compare ordered data/response records and per-target address counts.                               |
| axi_lite_slave vs axi_lite_regs    | 32-bit words; four RW words and one RO word; same byte strobes.           | 96 cases = 16 WSTRB masks $\times$ 3 AW/W orders $\times$ 2 response-stall settings.               |
| Additional register cases          | RO write, unmapped access, reset and post-reset access.                   | Same access-class response and no forbidden state update; each side checked against expected data. |

| Difference / boundary | Interpretation                                                                                                                                                                                              |
|-----------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Latency               | Not cycle-identical; compare completed transactions and protocol compliance.                                                                                                                                |
| Unmapped read data    | Our front-end returns zero; pinned PULP axi_lite_regs returns 0xBA5E1E55. Both return SLVERR; error-data policy is documented separately.                                                                   |
| Address aliases       | Different register-bank sizing; equality claimed only for the explicit compared address profile.                                                                                                            |
| AXI4-Full             | The PULP run does not establish equivalence of the DMA or soc_axi_full2lite to every upstream AXI4 module.                                                                                                  |
| Converter             | Local soc_tb_full2lite and soc_tb_full2lite_err check supported burst handling and errors. Local write error aggregation preserves DECERR over SLVERR; PULP is not a cycle-exact reference for that policy. |
| Versions              | axi: 70b8e54fd460e3308e58be596ceb3566a6e3576e; common_cells: 03d98106aa19952a10360d2230def85144a0008b.                                                                                                      |

Reference: <https://github.com/pulp-platform/axi>.